

Problem Solving Using C (25VBT0C103)



www.onlinejain.com

Module: 04

File Systems, Command-Line
Tools, & Build Automation

Module Information

Field	Details
Course Title	Problem Solving using C
Course Code	25VBT0C103
Module Number	Module 4 of 5
Module Title	File Systems, Command-Line Tools, and Build Automation

Module Overview

In the journey of becoming a proficient C programmer, writing code that works on screen is only one part of the story. Real-world software must persist data, respond to user inputs from the terminal, compile efficiently, and collaborate with teams through version-controlled repositories. Module 4 takes you from isolated programs to interconnected, production-ready C development.

This module opens with file handling — one of the most fundamental capabilities in any programming language. You will learn how C reads from and writes to text and binary files using the standard library functions `fopen`, `fclose`, `fprintf`, `fscanf`, `fread`, and `fwrite`. These skills are essential for building applications such as log management systems, data exporters, and configuration readers.

Next, you will explore command-line arguments, which allow programs to accept user input directly from the terminal without hard-coding values. This makes programs flexible, scriptable, and deployable in automated environments — a technique used widely in tools like `git`, `gcc`, and `ls`.

The module then introduces preprocessor directives, which are instructions processed before compilation. You will learn about `#define` for constants and macros, `#include` for importing headers, and conditional compilation using `#ifdef`/`#ifndef` — tools that help write portable, configurable, and reusable code.

Header files and modular programming are next — a critical skill for organising large C projects into logical, maintainable units. You will learn how to separate function declarations from implementations, avoid duplicate inclusions, and build clean module boundaries.

Build automation with Makefiles follows. Rather than typing `gcc` commands manually, a Makefile encodes the entire build process. You will understand targets, dependencies, rules, and variables so that a single `make` command can compile your entire project correctly and efficiently.

Version control with Git is an indispensable tool in modern software development. You will learn the fundamentals of Git — initialising repositories, staging, committing, branching, merging, and pushing to remote servers like GitHub or GitLab — with a focus on how teams at companies like Infosys, Wipro, and TCS manage code collaboratively.

Compiler flags and debugging with GDB bring you closer to the underlying machine. You will explore optimisation flags (-O2, -O3), warning flags (-Wall, -Wextra), and symbol inclusion (-g), then step through real program bugs using the GNU Debugger (GDB).

The module concludes with a brief introduction to Continuous Integration and Continuous Deployment (CI/CD) for C projects — illustrating how automated testing and build pipelines (using tools like GitHub Actions) form the backbone of professional C development workflows.

Learning Objectives

By the end of this module, you will be able to:

- Apply file handling functions to read from and write to text and binary files in C.
- Use command-line arguments to build flexible, configurable C programs.
- Implement preprocessor directives and macros to write portable and maintainable code.
- Organise a C project using header files and modular programming principles.
- Automate the build process using Makefiles and integrate Git for version control.

Learning Outcomes

After completing this module, you will be able to:

- Construct programs that persist data through text and binary file I/O operations.
- Analyse programs that accept argc/argv inputs to produce dynamic, flexible behaviour.
- Apply macro definitions and conditional compilation to control code compilation.
- Design modular C programs using header files and multi-file compilation.
- Set up a Git repository and a Makefile-based build system for a real-world C project.

Table of Topics

Sub-Unit	Topic	Description
4.1	Introduction to File Handling	Overview of file types, streams, and the FILE pointer in C
4.2	Text File Operations	fopen, fclose, fprintf, fscanf, fgetc, fputc, feof
4.3	Binary File Operations	fread, fwrite, fseek, ftell, rewind — structured data persistence
4.4	Command-Line Arguments	argc, argv, parsing input from the terminal
4.5	Preprocessor Directives	#define, #include, macros, conditional compilation (#ifdef)
4.6	Header Files and Modular Programming	Creating .h files, include guards, multi-file compilation
4.7	Build Automation with Makefiles	Targets, dependencies, variables, phony targets
4.8	Version Control with Git	init, add, commit, branch, merge, push, pull
4.9	Compiler Flags and Debugging with GDB	gcc flags, gdb commands, breakpoints, watchpoints
4.10	Introduction to CI/CD for C Projects	GitHub Actions workflow, automated build and test for C

4.1 Introduction to File Handling in C

Every program you have written so far lives only in memory — the moment it exits, all data is lost. File handling solves this fundamental problem by enabling programs to read from and write to persistent storage on disk. Whether it is saving a student's marks, logging a transaction at a POS terminal, or storing configuration for a server, files are the bridge between a running program and the real world.

In C, file operations are handled through the standard input/output library `<stdio.h>`. The central abstraction is the `FILE` pointer — a structure that holds metadata about an open file, including its current position, any buffering state, and error flags. You never manipulate the `FILE` structure directly; instead, you pass its pointer to library functions.

Types of Files

C recognises two broad categories of files:

- **Text Files** — Store data as human-readable ASCII or UTF-8 characters. Line endings and special characters may be translated depending on the operating system. Examples: `.txt`, `.csv`, `.log` files used in bank statement exports, attendance records, and application logs.
- **Binary Files** — Store data in raw memory format, exactly as it appears in RAM. No character translation occurs. Examples: `.bin`, `.dat`, `.obj` files used in compiled object code, image storage, and financial databases where speed and compactness matter.

The FILE Pointer

When you open a file, C returns a pointer of type `FILE *`. This pointer is the handle you use for all subsequent read, write, and close operations. If the file cannot be opened (e.g., it does not exist, or permissions are denied), the function returns `NULL`.

```
#include <stdio.h>

int main() {

    FILE *fp;                // Declare a FILE pointer

    fp = fopen("data.txt", "r"); // Open for reading

    if (fp == NULL) {

        printf("Error: Cannot open file.\n");
        return 1;

    }

    // File operations go here

    fclose(fp);              // Always close when done

    return 0;

}
```

File Opening Modes

Mode	Meaning	Behaviour
"r"	Read (text)	Opens existing file; error if file does not exist
"w"	Write (text)	Creates new file or truncates existing; overwrites
"a"	Append (text)	Writes at end; creates file if absent
"rb"	Read (binary)	Same as r but in binary mode
"wb"	Write (binary)	Same as w but in binary mode
"r+"	Read+Write	File must exist; no truncation
"w+"	Read+Write	Truncates or creates

💡 Did You Know?

The standard streams `stdin`, `stdout`, and `stderr` are themselves `FILE *` pointers opened automatically when a C program starts. That is why `fprintf(stdout, "Hello")` is equivalent to `printf("Hello")` — they use the same underlying mechanism. Many embedded systems at companies like Texas Instruments India use `fprintf(stderr)` to log diagnostic output separately from normal program output.

4.2 Text File Operations

Text file operations are the most commonly used form of file I/O in C. They work with human-readable data — perfect for log files, reports, configuration files, and data exports. The C standard library provides a rich set of functions for reading and writing text files, all working through the `FILE *` pointer.

Writing to a Text File — `fprintf` and `fputs`

`fprintf` works exactly like `printf`, but sends output to a file instead of the console. This makes it ideal for generating structured text output such as CSV exports, invoices, or audit logs.

```
/* Scenario: HDFC Bank — Write daily transaction log */  
  
#include <stdio.h>  
  
int main() {  
  
    FILE *fp = fopen("transaction_log.txt", "w");  
  
    if (fp == NULL) { printf("Cannot open log file.\n"); return 1;  
    }  
  
    fprintf(fp, "Date,Account,Type,Amount\n");  
}
```

```
fprintf(fp, "2024-03-15,SB0012345,Debit,5000.00\n");  
fprintf(fp, "2024-03-15,SB0012346,Credit,12500.50\n");  
  
fclose(fp);  
  
printf("Log written successfully.\n");  
  
return 0;  
  
}
```

fputs writes a single string to a file without a format string. It is faster when you have a pre-built string and no formatting is needed.

Reading from a Text File — fscanf and fgets

fscanf reads formatted data from a file, similar to scanf from stdin. fgets reads an entire line (up to a maximum number of characters), making it safer for reading variable-length lines from files.

```
/* Scenario: Read employee records from a CSV file — Infosys HR  
System */  
  
#include <stdio.h>  
  
int main() {  
    FILE *fp = fopen("employees.csv", "r");  
    if (fp == NULL) { printf("File not found.\n"); return 1; }  
  
    char line[200];  
    while (fgets(line, sizeof(line), fp) != NULL) {  
        printf("Record: %s", line);  
    }  
  
    fclose(fp);  
}
```

```
    return 0;
}
```

Checking End of File — feof

The `feof()` function returns a non-zero value when the end-of-file indicator is set. It is commonly used in while loops to read until the file is exhausted. However, a best practice is to check the return value of `fgets` or `fscanf` directly rather than relying solely on `feof`, since `feof` only reports EOF after a failed read attempt.

```
/* Read numbers from a file until EOF */
#include <stdio.h>

int main() {
    FILE *fp = fopen("scores.txt", "r");

    int score; int total = 0; int count = 0;

    while (fscanf(fp, "%d", &score) == 1) {
        total += score;
        count++;
    }

    printf("Average Score: %.2f\n", (float)total / count);
    fclose(fp);
    return 0;
}
```

🔍 Analysing Misconceptions: Text vs Binary Modes

✗ MYTH: Text mode and binary mode produce identical output on Linux/macOS, so there is no difference.

✓ REALITY: On Windows, text mode translates `\n` to `\r\n` (CRLF) on write and back on read. Binary mode performs no translation. If you open a binary file (like a .jpg image) in text mode, this translation can corrupt the data. Always match the mode to the file type.

✓ Let's Check

Q.No	Type	Question
1	MCQ	Which function is used to write formatted data to a file in C?
2	MCQ	What does <code>fopen()</code> return if the file cannot be opened?
3	Fill in Blank	The function _____ reads a line from a file into a character array.
4	Fill in Blank	The file mode _____ opens an existing file for reading and writing without truncating it.
5	True/False	<code>fclose()</code> must be called after completing file operations to flush the buffer and release the file.
6	True/False	The "w" mode in <code>fopen()</code> will fail if the file already exists.

Answers:

Q.No	Answer
1	a) fprintf()
2	b) NULL
3	fgets()
4	"r+"
5	True — fclose() flushes any buffered data and releases the file handle back to the OS.
6	False — "w" mode truncates and overwrites an existing file; it does not fail.

4.3 Binary File Operations

While text files store data in human-readable form, binary files store data exactly as it exists in memory — raw bytes with no conversion. This makes binary I/O significantly faster and more compact, especially when dealing with structured data such as records, images, or serialised objects.

Binary file handling is widely used in industry — from storing patient biometric records at Apollo Hospitals to persisting sensor readings on IoT devices at Tata Power's smart grid. Understanding binary I/O is essential for systems programming, embedded development, and high-performance data pipelines.

Writing Binary Data — fwrite

`fwrite(ptr, size, count, fp)` writes `count` elements, each of `size` bytes, from the memory address `ptr` into the file `fp`. It returns the number of elements successfully written.

```
/* Scenario: Store student exam records in binary format - DU  
Examination System */
```

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int rollNo;
    char name[50];
    float marks;
} Student;

int main() {
    Student s1 = {1001, "Ananya Sharma", 87.5};
    Student s2 = {1002, "Rohan Mehta", 91.0};

    FILE *fp = fopen("students.dat", "wb");
    if (!fp) { printf("Error opening file.\n"); return 1; }

    fwrite(&s1, sizeof(Student), 1, fp);
    fwrite(&s2, sizeof(Student), 1, fp);

    fclose(fp);
    printf("Records saved.\n");
    return 0;
}
```

Reading Binary Data — fread

fread(ptr, size, count, fp) reads count elements of size bytes each from fp into the buffer at ptr. It returns the number of elements successfully read.

```
/* Read student records back from binary file */
#include <stdio.h>

typedef struct { int rollNo; char name[50]; float marks; } Student;

int main() {
    Student s;

    FILE *fp = fopen("students.dat", "rb");

    if (!fp) { printf("File not found.\n"); return 1; }

    while (fread(&s, sizeof(Student), 1, fp) == 1) {
        printf("Roll: %d | Name: %s | Marks: %.1f\n", s.rollNo,
s.name, s.marks);
    }

    fclose(fp);
    return 0;
}
```

Random Access — fseek, ftell, rewind

Unlike sequential access where you always read from start to end, random access lets you jump to any position in a file. This is critical for database-style applications where you need to read or update a specific record without processing all preceding records.

- `fseek(fp, offset, origin)` — Moves the file position cursor. `SEEK_SET` = from start, `SEEK_CUR` = from current, `SEEK_END` = from end.
- `ftell(fp)` — Returns the current position of the cursor (in bytes from start).

- `rewind(fp)` — Moves cursor back to the beginning of the file (equivalent to `fseek(fp, 0, SEEK_SET)`).

```
/* Update a specific student record by roll number */
#include <stdio.h>

typedef struct { int rollNo; char name[50]; float marks; } Student;

int main() {
    FILE *fp = fopen("students.dat", "rb+");
    Student s; int target = 1002;

    while (fread(&s, sizeof(Student), 1, fp) == 1) {
        if (s.rollNo == target) {
            s.marks = 95.0;    // Update marks
            fseek(fp, -(long)sizeof(Student), SEEK_CUR);
            fwrite(&s, sizeof(Student), 1, fp);
            printf("Record updated for Roll %d\n", target);
            break;
        }
    }

    fclose(fp);
    return 0;
}
```


💡 Did You Know?

MySQL and PostgreSQL — both heavily used by Indian enterprises like Reliance Retail, Ola, and IRCTC — store table data in binary format (.ibd and .bin files respectively). The record layout and fseek-style random access principles you learn in C directly underlie how these database engines read and write data pages on disk.

4.4 Command-Line Arguments

So far, every program has had its inputs hard-coded or entered interactively through `scanf`. Command-line arguments offer a third way: passing inputs directly when launching the program from the terminal. This makes programs scriptable, configurable, and usable in automated pipelines — a cornerstone of Unix philosophy and modern DevOps workflows.

When you run a command like `gcc -o output main.c`, the flags `-o`, `output`, and `main.c` are command-line arguments passed to the `gcc` program. Your own C programs can accept arguments in exactly the same way.

The `main()` Signature with Arguments

The extended signature of `main` accepts two parameters:

- `int argc` — Argument Count: the total number of arguments including the program name itself.
- `char *argv[]` — Argument Vector: an array of strings where `argv[0]` is the program name, `argv[1]` is the first argument, and so on.

```
/* Basic argc/argv demonstration */
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Program name: %s\n", argv[0]);
    printf("Number of arguments: %d\n", argc - 1);

    for (int i = 1; i < argc; i++) {
        printf("Argument %d: %s\n", i, argv[i]);
    }

    return 0;
}
```

Running: ./myprogram hello world 42

Output: Program name: ./myprogram | Argument 1: hello | Argument 2: world |
Argument 3: 42

Practical Example: File Converter Tool

Consider a real-world scenario at Wipro Technologies, where a data migration tool reads a source file and writes its contents with a line count prefix. The file names are passed as command-line arguments for flexibility across deployments.

```
/* Wipro Data Migration Tool - accepts input and output filenames
as arguments */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
```

```
        printf("Usage: %s <input_file> <output_file>\n", argv[0]);
        return 1;
    }

    FILE *in  = fopen(argv[1], "r");
    FILE *out = fopen(argv[2], "w");

    if (!in || !out) { printf("File error.\n"); return 1; }

    char line[256]; int lineNum = 0;
    while (fgets(line, sizeof(line), in)) {

        fprintf(out, "%4d: %s", ++lineNum, line);

    }

    printf("Processed %d lines.\n", lineNum);
    fclose(in); fclose(out);
    return 0;
}
```

Converting String Arguments to Numbers

All argv values are strings. Use atoi() for integers and atof() for floating-point conversion from <stdlib.h>.

```
#include <stdio.h>

#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 3) { printf("Usage: calc <num1> <num2>\n"); return
1; }
```

```
int a = atoi(argv[1]);  
int b = atoi(argv[2]);  
printf("%d + %d = %d\n", a, b, a + b);  
return 0;  
}
```

🔍 Analysing Misconceptions: argv Values Are Integers

✗ MYTH: If you pass 42 as a command-line argument, argv[1] holds the integer 42.

✓ REALITY: argv is always an array of strings (char *). Even numeric arguments arrive as character strings. You must explicitly convert using atoi(), atof(), or strtol() before performing arithmetic. Forgetting this conversion is a common cause of bugs in beginner C tools.

4.5 Preprocessor Directives

Before your C code reaches the compiler, it passes through the C Preprocessor — a text-substitution tool that processes instructions beginning with #. These directives are not C statements; they are meta-instructions that control how the source code is prepared for compilation. Understanding them is essential for writing portable, configurable, and reusable C code.

The preprocessor is invoked automatically as the first stage of the gcc compilation pipeline, before tokenising or parsing begins.

#include — File Inclusion

#include inserts the contents of another file at the point of inclusion. There are two forms:

- #include <stdio.h> — Searches the system include directories (standard library headers).

- `#include "myheader.h"` — Searches the current directory first, then system directories.

Every time you use `printf`, you are relying on `#include <stdio.h>` to make the declaration of `printf` available to the compiler.

#define — Constants and Macros

`#define` creates simple text substitutions. Before compilation, every occurrence of the identifier is replaced by its defined value.

```
/* Scenario: BESCOM Electricity Bill Calculator */
#include <stdio.h>

#define RATE_PER_UNIT 5.50 /* Cost per kWh in rupees */
#define FIXED_CHARGE 75.00 /* Monthly fixed charge */
#define TAX_RATE 0.18 /* GST 18% */

int main() {
    float units;

    printf("Enter units consumed: ");
    scanf("%f", &units);

    float bill = units * RATE_PER_UNIT + FIXED_CHARGE;
    float tax = bill * TAX_RATE;
    printf("Bill: Rs. %.2f | GST: Rs. %.2f | Total: Rs. %.2f\n",
           bill, tax, bill + tax);
    return 0;
}
```

Parameterised Macros

Macros can take arguments, acting like inline functions. They are expanded at compile time and have no function-call overhead.

```
#define MAX(a, b)      ((a) > (b) ? (a) : (b))
#define SQUARE(x)      ((x) * (x))
#define ABS(x)         ((x) < 0 ? -(x) : (x))

int main() {
    printf("Max of 10, 20: %d\n", MAX(10, 20));    // 20
    printf("Square of 7:  %d\n", SQUARE(7));      // 49
    printf("Abs of -15:   %d\n", ABS(-15));       // 15
    return 0;
}
```

Conditional Compilation — #ifdef, #ifndef, #endif

Conditional compilation allows different code paths to be compiled based on whether a macro is defined. This is invaluable for platform-specific code, debug builds, and feature flags.

```
#include <stdio.h>

#define DEBUG    /* Comment out to disable debug output */

int main() {
    int balance = 15000;

#ifdef DEBUG
    printf("[DEBUG] Balance before deduction: %d\n", balance);
#endif
}
```

```
    balance -= 5000;

    printf("Remaining balance: Rs. %d\n", balance);

    return 0;
}
```

💡 Did You Know?

The Linux kernel source code — which runs on servers at ISRO, Indian Railways, and most cloud providers — uses thousands of `#define` macros and conditional compilation blocks. Platform-specific code for ARM (used in smartphones), x86 (used in PCs), and RISC-V is selected at compile time using `#ifdef` guards like `CONFIG_ARM`, `CONFIG_X86`, etc.

✓ Let's Check

Q.No	Type	Question
1	MCQ	Which preprocessor directive is used to define a symbolic constant in C?
2	MCQ	What is the correct syntax to include a user-defined header file?
3	Fill in Blank	<code>#ifdef</code> compiles a block of code only if the specified macro is ____.
4	Fill in Blank	The macro <code>#define SQUARE(x) ((x)*(x))</code> will expand <code>SQUARE(5)</code> to ____.

5	True/False	A parameterised macro can replace a simple function with no arguments passed by reference.
6	True/False	Preprocessor directives are processed before the C compiler tokenises the source code.

Answers:

Q.No	Answer
1	b) #define
2	c) #include "filename.h"
3	defined
4	((5)*(5)) which evaluates to 25
5	True — macros are textual substitutions and do not have real parameters; however, they cannot take arguments by reference.
6	True — the preprocessor runs before the compilation stage.

4.6 Header Files and Modular Programming

As programs grow in size and complexity, putting everything into a single .c file becomes unmanageable. Modular programming is the practice of dividing a program into separate, self-contained units — each responsible for a specific aspect of functionality. In C, this is achieved through header files (.h) and multiple source files (.c), compiled and linked together.

This approach is used universally in professional C projects. The GNU C Library (glibc), which underpins every Linux system, is organised into hundreds of modules — each with its own .h declaration file and .c implementation file. Companies like Samsung India, Qualcomm India, and Bosch India follow the same modular pattern in embedded software.

What Goes in a Header File?

A header file contains declarations, not definitions. It tells the compiler what exists, while the .c file contains the actual code.

- Function prototypes (declarations)
- Struct and typedef definitions
- #define constants and macros
- extern variable declarations

Include Guards

If a header file is included from multiple places in a project, the compiler may process it multiple times, causing duplicate declaration errors. Include guards prevent this using conditional compilation:

```
/* File: account.h */

#ifndef ACCOUNT_H    /* If ACCOUNT_H is not defined... */
#define ACCOUNT_H    /* ...define it now (so this block runs only
once) */

#define MAX_NAME 50

typedef struct {
    int accNumber;
    char holder[MAX_NAME];
    double balance;
} Account;

void deposit(Account *a, double amount);
void withdraw(Account *a, double amount);
void printAccount(const Account *a);

#endif /* ACCOUNT_H */
```

The Implementation File

```
/* File: account.c – Implementation */

#include <stdio.h>

#include "account.h"

void deposit(Account *a, double amount) {
    a->balance += amount;

    printf("Deposited Rs. %.2f | New Balance: Rs. %.2f\n", amount,
a->balance);
}
```

```
}

void withdraw(Account *a, double amount) {
    if (amount > a->balance) { printf("Insufficient funds.\n");
return; }

    a->balance -= amount;

    printf("Withdrew Rs. %.2f | New Balance: Rs. %.2f\n", amount,
a->balance);
}

void printAccount(const Account *a) {
    printf("Account: %d | Holder: %s | Balance: Rs. %.2f\n",
        a->accNumber, a->holder, a->balance);
}
```

The Main File

```
/* File: main.c - Entry point */

#include <stdio.h>
#include "account.h"

int main() {

    Account myAcc = {100201, "Priya Nair", 25000.00};
    printAccount(&myAcc);
    deposit(&myAcc, 5000.00);
    withdraw(&myAcc, 8000.00);
    printAccount(&myAcc);
    return 0;
}
```

Compiling Multiple Files

To compile a multi-file C project from the terminal:

```
# Compile each .c file to an object file, then link
gcc -c account.c -o account.o
gcc -c main.c -o main.o
gcc account.o main.o -o bank_app
./bank_app
```

This separation means that if you only change `account.c`, you only recompile that file — saving build time. This is exactly what Makefiles automate, as you will see in the next sub-unit.

4.7 Build Automation with Makefiles

As a project grows from 2 files to 20 files, manually typing `gcc` commands becomes error-prone and time-consuming. A Makefile automates the build process by defining exactly how each file is compiled and in what order — and crucially, only recompiling files that have changed. This is the same technology used to build the Linux kernel, GCC itself, and the embedded software running on millions of Indian smartphones and IoT devices.

The `make` utility reads a Makefile and executes only the minimum set of commands needed to bring the project up to date.

Basic Makefile Structure

A Makefile consists of rules. Each rule has three parts:

```
target: dependencies
    recipe (must be indented with a TAB, not spaces)
```

- **target** — The file to be built (or an action name like `clean`).

- dependencies — Files the target depends on; if any dependency is newer than the target, the recipe runs.
- recipe — The shell command to execute (e.g., a gcc invocation).

A Complete Makefile Example — Bank Application

```
# Makefile for the SBI Bank Application

CC      = gcc
CFLAGS  = -Wall -Wextra -g
TARGET  = bank_app

# Default target — build the application
all: $(TARGET)

$(TARGET): main.o account.o
    $(CC) $(CFLAGS) main.o account.o -o $(TARGET)

main.o: main.c account.h
    $(CC) $(CFLAGS) -c main.c -o main.o

account.o: account.c account.h
    $(CC) $(CFLAGS) -c account.c -o account.o

# Phony target — not a real file, always runs
.PHONY: clean

clean:
    rm -f *.o $(TARGET)
```

With this Makefile, simply running `make` in the project directory builds the entire application. Running `make clean` removes all compiled objects and the executable.

Makefile Variables

Variable	Purpose
CC	C compiler to use (e.g., gcc, clang)
CFLAGS	Compiler flags applied to all compilations
LDFLAGS	Linker flags (e.g., -lm to link math library)
TARGET	Name of the final executable
\$@	Automatic variable — expands to the target name
\$<	Automatic variable — expands to the first dependency
\$^	Automatic variable — expands to all dependencies

Analysing Misconceptions: Makefiles Use Spaces for Indentation

✗ MYTH: You can use spaces or tabs to indent recipe lines in a Makefile.

✓ REALITY: Makefiles strictly require a TAB character (not spaces) to indent recipe lines. This is one of the most common errors beginners make. If you use spaces, `make` will report "missing separator" errors. Configure your editor to insert literal tabs in `.mk` and Makefile files.

4.8 Version Control with Git

Every professional software project needs version control — a system that tracks changes to source code over time, allows multiple developers to collaborate without overwriting each other's work, and provides a full history of every change ever made. Git is the world's most widely used version control system, and it forms the backbone of collaboration at companies like Infosys, TCS, Wipro, HCL, and virtually every technology startup in India.

Understanding Git is no longer optional for a software professional. Whether you are a solo developer tracking your own changes or part of a 1,000-person engineering team at a major IT firm, Git provides the infrastructure for safe, auditable, and collaborative development.

Core Git Concepts

- Repository (Repo) — A directory tracked by Git, containing your project files and a hidden .git folder that stores the entire history.
- Working Directory — The local folder where you edit files.
- Staging Area (Index) — A preparation area where you add changes before committing.
- Commit — A permanent snapshot of the staged changes, along with a message describing what changed.
- Branch — An independent line of development; the default is called main (formerly master).
- Remote — A version of the repository hosted on a server (e.g., GitHub, GitLab, Bitbucket).

Essential Git Commands

```
# Initialise a new repository
git init
```

Check current status of files git status
Stage files for commit
git add main.c account.c account.h Makefile
git add . # Stage everything
Commit with a descriptive message
git commit -m "feat: add account deposit and withdraw functions"
View commit history
git log --oneline
Create and switch to a new branch git checkout -b feature/transaction-log
Merge a branch back into main git checkout main git merge feature/transaction-log
Connect to a remote repository git remote add origin https://github.com/username/bank-app.git
Push to remote
git push -u origin main
Pull latest changes git pull origin main

A Typical Git Workflow at an Indian IT Company

At a company like Infosys, developers follow a feature-branch workflow:

- Step 1 — `git pull`: Fetch the latest changes from the remote main branch to avoid conflicts.
- Step 2 — `git checkout -b feature/your-feature`: Create a new branch for your feature or bug fix.
- Step 3 — Edit files, then `git add` and `git commit`: Stage and commit your changes with clear, descriptive messages.
- Step 4 — `git push origin feature/your-feature`: Push the branch to the remote repository.
- Step 5 — Raise a Pull Request (PR): Request a code review from peers before merging into main.
- Step 6 — After approval, merge: The branch is merged and the feature is now part of the main codebase.

💡 Did You Know?

GitHub alone hosts over 420 million repositories as of 2024 (GitHub, 2024). Indian developers are among the top contributors globally, with over 13 million accounts — a number growing by roughly 2 million annually. Companies like Freshworks, Razorpay, and PhonePe open-source significant parts of their infrastructure code on GitHub, using the same Git workflows you are learning here.

 Let's Check

Q.No	Type	Question
1	MCQ	Which Git command stages all modified files in the current directory for the next commit?
2	MCQ	What does git log --oneline display?
3	Fill in Blank	A Git _____ is a pointer to a specific commit, enabling parallel development lines.
4	Fill in Blank	The git _____ command fetches and merges changes from a remote repository.
5	True/False	git commit saves changes directly to the remote repository.
6	True/False	git merge integrates changes from one branch into another.

Answers:

Q.No	Answer
1	b) git add .
2	a) A condensed one-line summary of each commit with its hash
3	branch
4	pull
5	False — git commit saves changes to the local repository. git push is required to upload to a remote.
6	True — git merge combines the histories of two branches.

4.9 Compiler Flags and Debugging with GDB

Writing code that compiles is just the beginning. In professional development, code must be correct, efficient, and robust. Compiler flags let you control how gcc translates your source code — from how aggressively it optimises to how many warnings it reports. The GNU Debugger (GDB) then allows you to step through a running program line by line, inspect variables, and pinpoint the exact cause of a crash or logic error.

These skills are used daily at system software companies, embedded teams, and kernel developers across India. Knowing how to compile with the right flags and debug with GDB can reduce hours of guesswork to minutes of systematic investigation.

Important GCC Compiler Flags

Flag	Effect
-Wall	Enable all common warnings (unused variables, implicit declarations, etc.)
-Wextra	Enable extra warnings beyond -Wall
-Werror	Treat all warnings as errors — forces clean, warning-free code
-g	Include debug symbols — required for GDB to map machine code to source lines
-O0	No optimisation (default) — best for debugging; code behaves as written
-O2	Moderate optimisation — common for production builds
-O3	Aggressive optimisation — maximum performance; may reorder or inline code
-std=c99	Compile using the C99 standard

-std=c11	Compile using the C11 standard
-o <name>	Specify output file name

A typical development build command:

```
gcc -Wall -Wextra -g -std=c11 -o bank_app main.c account.c
```

Debugging with GDB

GDB (GNU Debugger) is a command-line debugger that lets you run a program under controlled conditions. You can pause execution at any line, inspect variables, step through code one line at a time, and diagnose crashes (segmentation faults).

Always compile with `-g` before using GDB — without debug symbols, GDB cannot map instructions back to your source lines.

```
# Compile with debug symbols
gcc -g -Wall -o bank_app main.c account.c

# Launch GDB
gdb ./bank_app
```

Essential GDB Commands

GDB Command	What It Does
run (r)	Start the program
break <line> (b)	Set a breakpoint at a line number or function name
next (n)	Execute the next line (step over function calls)

step (s)	Step into function calls
print <var> (p)	Print the current value of a variable
backtrace (bt)	Show the call stack — useful after a crash
continue (c)	Continue running until the next breakpoint
watch <var>	Pause execution whenever a variable changes value
quit (q)	Exit GDB

```
(gdb) break main      # Set breakpoint at start of main
(gdb) run              # Start the program
(gdb) next             # Execute next line
(gdb) print balance    # Print value of variable balance
(gdb) backtrace        # Show how we got here (call stack)
(gdb) continue         # Run until next breakpoint
(gdb) quit             # Exit debugger
```

🔍 Analysing Misconceptions: Warnings Are Optional

✗ MYTH: Compiler warnings are just suggestions and can be safely ignored if the program compiles.

✓ REALITY: Warnings often signal real bugs — uninitialized variables, signed/unsigned mismatches, implicit function declarations, and potential buffer overflows. At companies like Qualcomm India and Intel India, CI/CD pipelines reject code with -Wall -Wextra warnings. Enabling -Werror makes this enforced at compile time. Ignoring warnings is one of the most common causes of subtle runtime bugs.

4.10 Introduction to CI/CD for C Projects

Continuous Integration (CI) and Continuous Deployment (CD) are modern software engineering practices that automate the building, testing, and deployment of code. Every time a developer pushes changes to a repository, the CI/CD pipeline automatically compiles the code, runs tests, and reports whether the changes break anything. This ensures that software is always in a releasable state.

While CI/CD is often associated with web development, it is equally important for C projects — especially in embedded systems, device drivers, and server software. Indian technology leaders like HCL Technologies, L&T Technology Services, and KPIT Technologies routinely implement CI/CD pipelines for C and C++ codebases.

The CI/CD Pipeline for a C Project

- Developer pushes code to GitHub/GitLab repository.
- CI system detects the push and automatically triggers the pipeline.
- Build stage: Compile the code using gcc or the project Makefile.
- Test stage: Run unit tests (e.g., using a test framework like Unity or Check).
- Lint stage: Run static analysis tools like cppcheck or clang-tidy to catch code quality issues.
- Report: Pass or Fail result is posted back to the repository, visible on the Pull Request.
- Deploy stage (CD): If all checks pass, automatically deploy to a staging or production environment.

GitHub Actions — A CI/CD Workflow for C

GitHub Actions uses YAML workflow files stored in `.github/workflows/`. Here is a minimal CI workflow that compiles and tests a C project on every push:

```
# .github/workflows/build.yml
name: Build and Test C Project

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v3

      - name: Install GCC
        run: sudo apt-get install -y gcc

      - name: Build project
        run: make all

      - name: Run tests
        run: ./run_tests.sh

      - name: Static analysis
        run: cppcheck --enable=all --error-exitcode=1 .
```

With this setup, every git push to GitHub triggers an automated build and test cycle. If any step fails, the developer is immediately notified — preventing broken code from reaching production.

Static Analysis Tools for C

Tool	Purpose
cppcheck	Open-source static analyser; finds memory leaks, null pointer dereferences, buffer overflows
clang-tidy	Clang-based linter; enforces code style and catches common C bugs
Valgrind	Runtime memory error detector; finds leaks, use-after-free, invalid reads/writes
SonarQube	Enterprise-grade code quality platform; used by Infosys and TCS in CI pipelines
gcov / lcov	Code coverage tools; measure how much of your code is covered by tests

💡 Did You Know?

The Mars Ingenuity Helicopter — the first powered aircraft to fly on another planet (2021) — runs software written in C and C++. NASA JPL uses rigorous CI/CD pipelines, static analysis (including cppcheck and custom tools), and automated test suites to validate every line of code before it is deployed. Similar practices are followed at ISRO for satellite software and at DRDO for defence systems.

■ Case Study: Tata Consultancy Services (TCS) — Automated Build Pipeline for a C-Based Embedded Billing System

Background:

Tata Consultancy Services (TCS), one of India's largest IT services companies, manages the software development lifecycle for a major power distribution utility in Maharashtra. The project involves a C-based embedded firmware for smart electricity meters deployed across 8 lakh households. The firmware handles binary file-based tamper logs, reads command-line configuration at boot, and is compiled with strict quality gates. (Based on publicly available TCS embedded practice case studies, 2023.)

Problem:

Before 2022, the team of 24 developers manually compiled firmware with hand-typed gcc commands across three different Linux environments. Integration errors were only discovered during end-of-sprint testing, resulting in 2–3 day delays per sprint. A critical tamper log binary file corruption bug — caused by mixing text and binary file modes — escaped into production, affecting 12,000 meters and requiring a field update at significant cost.

Solution:

The team implemented a complete modular build system with the following components:

- Modular C architecture with separate header files for `meter_io.h`, `tamper_log.h`, and `config.h` — enabling independent compilation and testing.
- A unified Makefile with `CC=arm-linux-gnueabi-gcc` for cross-compilation, `-Wall -Werror` enforced in `CFLAGS`, and phony clean and test targets.

- Binary file I/O with strict "rb"/"wb" mode enforcement via a custom macro `OPEN_BINARY(path, mode)`, preventing the text/binary mode confusion from recurring.
- Command-line argument parsing for meter ID, region code, and firmware version — replacing hard-coded configuration values.
- A GitHub Actions CI/CD pipeline that compiled the project on every pull request, ran a suite of 47 unit tests using the Unity test framework, and ran `cppcheck` static analysis with zero-warning enforcement.

Results:

Within two sprints of adopting the new pipeline: Integration defect rate fell by 78%. Build time for the complete firmware reduced from 4 minutes (manual) to 45 seconds (automated Makefile). The binary file corruption class of bugs was eliminated entirely due to automated mode checking. The CI/CD pipeline caught 23 regressions before they reached the testing branch in the first quarter of operation. The smart meter project was subsequently certified under IEC 62056 (DLMS/COSEM) standard, in part due to the demonstrable code quality process.

Key C Concepts Demonstrated:

- Binary file I/O (`fopen` in "rb"/"wb" mode, `fwrite/fread` for tamper records)
- Command-line arguments (`argc/argv` for meter configuration at boot)
- Modular header files (`meter_io.h`, `tamper_log.h` with include guards)
- Makefile build automation with cross-compiler and phony targets
- Git version control and GitHub Actions CI/CD pipeline
- Compiler flags: `-Wall -Werror -g` for debug, `-O2` for release builds

Module Summary

Module 4 has taken you from writing standalone C programs to building professional, production-ready C software with persistent storage, flexible interfaces, automated builds, and version-controlled collaboration.

You began with file handling — the foundation of data persistence in C. Text file operations using `fprintf`, `fscanf`, and `fgets` allow programs to read and write human-readable data such as logs, reports, and configurations. Binary file operations using `fread`, `fwrite`, and `fseek` provide fast, compact storage of structured data — used in everything from university examination systems to IoT sensor platforms.

Command-line arguments (`argc/argv`) transform programs from fixed tools into flexible utilities that accept runtime configuration — the same mechanism used by `gcc`, `git`, and every Unix command-line tool.

Key takeaways from this module:

- Always close files with `fclose()` and check for `NULL` returns from `fopen()` to handle errors gracefully.
- Use text mode ("`r`", "`w`", "`a`") for human-readable data; binary mode ("`rb`", "`wb`") for structured records and raw data — never mix them.
- Command-line arguments arrive as strings; convert with `atoi()` or `atof()` before arithmetic use.
- Preprocessor directives (`#define`, `#include`, `#ifdef`) run before compilation and enable portable, configurable code.
- Header files with include guards and separate `.c` implementation files are the building blocks of modular, maintainable C projects.
- Makefiles encode the entire build process; a single `make` command is sufficient to compile a multi-file project correctly and efficiently.

-
- Git provides a complete version history, supports team collaboration through branches and pull requests, and is indispensable in every modern software organisation.
 - Compiler flags `-Wall`, `-Wextra`, and `-g` produce safer, more debuggable code; GDB allows you to step through running programs and inspect state at any point.
 - CI/CD pipelines automate build, test, and static analysis on every commit — the standard for quality-conscious teams.

Together, these skills form the professional toolkit that separates a hobbyist programmer from an industry-ready software developer. The ability to write correct, well-organised C code, manage it with Git, build it with a Makefile, and validate it with a CI/CD pipeline is expected at every major IT employer in India.

Glossary

Term	Definition
FILE pointer	A pointer to a FILE structure that serves as a handle for all file I/O operations in C.
fopen()	Opens a file in a specified mode and returns a FILE pointer; returns NULL on failure.
fwrite() / fread()	Functions for writing/reading raw binary data to/from a file in blocks of specified size.
fseek()	Moves the file position cursor to a specified offset — enabling random access in binary files.
argc / argv	Parameters of main() that receive the count and values of command-line arguments.
#define	Preprocessor directive that creates a symbolic constant or parameterised macro.
Include Guard	A #ifndef/#define/#endif pattern in header files that prevents duplicate inclusion.
Makefile	A build automation file that defines targets, dependencies, and compilation rules for make.
Git Repository	A directory tracked by Git, containing project files and a complete history of changes.
CI/CD Pipeline	An automated workflow that builds, tests, and analyses code on every code push to a repository.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

1. Which function is used to open a file for binary reading in C?
 - a) `fopen("file","r")`
 - b) `fopen("file","rb")`
 - c) `fopen("file","br")`
 - d) `open("file","rb")`
2. What does `fclose()` do?
 - a) Deletes the file
 - b) Rewinds to beginning
 - c) Flushes buffer and releases the file handle
 - d) Reads the last record
3. Which function reads a line of text from a file into a character array?
 - a) `fscanf()`
 - b) `fgetc()`
 - c) `fgets()`
 - d) `fread()`
4. What is the return value of `fopen()` when the file cannot be opened?
 - a) 0
 - b) -1
 - c) NULL
 - d) EOF
5. `fwrite(ptr, size, count, fp)` writes how many bytes to the file?
 - a) size only
 - b) count only
 - c) size + count

-
- d) size * count
6. Which fseek origin constant positions the cursor relative to the end of the file?
- a) SEEK_SET
 - b) SEEK_END
 - c) SEEK_CUR
 - d) SEEK_ABS
7. What is stored in argv[0] when a C program is executed?
- a) The first user argument
 - b) NULL
 - c) The program name/path
 - d) The number of arguments
8. Which function converts a command-line string argument to an integer?
- a) str2int()
 - b) stoi()
 - c) atoi()
 - d) intval()
9. What does the preprocessor directive #define MAX 100 do?
- a) Declares an integer variable MAX
 - b) Creates a constant using const
 - c) Replaces every occurrence of MAX with 100 before compilation
 - d) Allocates 100 bytes of memory
10. Which directive is used for conditional compilation based on whether a macro is defined?
- a) #if defined()
 - b) #ifdef
 - c) #check

d) #when

11. What is the purpose of an include guard in a header file?

- a) To speed up compilation
- b) To prevent the header from being included more than once
- c) To encrypt the header
- d) To limit the file to 100 lines

12. In a Makefile, what character must be used to indent recipe lines?

- a) Two spaces
- b) Four spaces
- c) A tab character
- d) A semicolon

13. Which Makefile variable conventionally holds the compiler flags?

- a) CC
- b) CFLAGS
- c) FLAGS
- d) OPTS

14. Which git command stages all modified and new files for the next commit?

- a) git commit -a
- b) git push .
- c) git add .
- d) git stage all

15. What does git checkout -b feature/login do?

- a) Deletes the feature/login branch
- b) Creates and switches to a new branch called feature/login
- c) Merges the feature/login branch into main
- d) Pushes the branch to remote

16. Which gcc flag includes debug symbol information needed by GDB?
- a) -debug
 - b) -symbols
 - c) -g
 - d) -D
17. Which GDB command shows the full call stack at the point of a crash?
- a) print stack
 - b) list
 - c) backtrace
 - d) trace call
18. What does the -Wall gcc flag do?
- a) Enables all optimisations
 - b) Enables all common compiler warnings
 - c) Compiles all .c files in the directory
 - d) Links against all system libraries
19. Which tool detects memory leaks and invalid memory accesses at runtime in C?
- a) cppcheck
 - b) GDB
 - c) Valgrind
 - d) clang-tidy
20. In a CI/CD pipeline, which stage typically runs after the build stage?
- a) Deployment
 - b) Commit
 - c) Test stage
 - d) Source checkout

Section B: True / False

State whether each statement is True or False.

1. The "a" mode in `fopen()` truncates an existing file before writing.
2. `ftell()` returns the current position of the file cursor in bytes from the beginning of the file.
3. In a Makefile, if none of the dependencies have changed, make will still re-run the recipe.
4. `git push` sends your local commits to a remote repository.
5. Compiling a C program with `-O3` always produces a faster program than `-O2` without any trade-offs.

Section C: Short Answer Questions

Answer each question in 4–8 sentences or with relevant code.

1. Write a C program that reads an integer from the command line and writes its multiplication table (1 to 10) to a text file named "table_N.txt" where N is the given number.
2. Explain the difference between text mode and binary mode in C file handling. In what scenarios should binary mode be preferred? Give a practical example from a real-world application.
3. Create a header file `math_utils.h` with include guards that declares two functions: `int factorial(int n)` and `float power(float base, int exp)`. Then write the corresponding `math_utils.c` with implementations.
4. Write a Makefile for a C project with the files `main.c`, `io_utils.c`, and `io_utils.h`. Include all, clean targets and use `-Wall -g` in `CFLAGS`. Explain each rule in the Makefile.
5. Describe the Git commands you would use to: (a) initialise a repository, (b) commit your Makefile project, (c) create a new branch called `feature/binary-log`, and (d) push that branch to a remote named `origin`. Write each command with a brief explanation.

Self Assessment — Answer Keys

MCQ

Q.No	Answer	Option Text
1	B	b) fopen("file","rb")
2	C	c) Flushes buffer and releases the file handle
3	C	c) fgets()
4	C	c) NULL
5	D	d) size * count
6	B	b) SEEK_END
7	C	c) The program name/path
8	C	c) atoi()
9	C	c) Replaces every occurrence of MAX with 100 before compilation
10	B	b) #ifdef
11	B	b) To prevent the header from being included more than once
12	C	c) A tab character
13	B	b) CFLAGS
14	C	c) git add .
15	B	b) Creates and switches to a new branch called feature/login
16	C	c) -g
17	C	c) backtrace
18	B	b) Enables all common compiler warnings

19	C	c) Valgrind
20	C	c) Test stage

True / False Answer Key

Q.No	Answer	Explanation
1	False	Incorrect — refer to the relevant sub-unit for the correct understanding.
2	True	Correct — this statement accurately describes the behaviour.
3	False	Incorrect — refer to the relevant sub-unit for the correct understanding.
4	True	Correct — this statement accurately describes the behaviour.
5	False	Incorrect — refer to the relevant sub-unit for the correct understanding.

Short Answer — Model Answers

Q.No	Model Answer
1	The program should use argc to check that exactly 2 arguments exist (argv[0] = program, argv[1] = N). Convert argv[1] using atoi(). Construct the filename using sprintf(filename, "table_%d.txt", n). Open the file with fopen(filename, "w"), use a for loop with fprintf(fp, "%d x %d = %d\n", n, i, n*i) for i = 1 to 10, then fclose(fp).
2	Text mode translates newline characters (\n to \r\n on Windows) and may interpret special byte sequences. Binary mode performs no translation —

	data is written and read exactly as it exists in memory. Binary mode is preferred for: (a) struct records (e.g., student or patient data), (b) image or audio files, (c) fixed-length database records requiring fseek-based random access. Example: Storing Apollo Hospital patient records as a struct array in "wb" mode ensures each record is exactly sizeof(Patient) bytes, enabling direct random access by record index.
3	<pre>math_utils.h: #ifndef MATH_UTILS_H / #define MATH_UTILS_H / int factorial(int n); / float power(float base, int exp); / #endif. math_utils.c: #include "math_utils.h" / int factorial(int n) { return (n <= 1) ? 1 : n * factorial(n-1); } / float power(float base, int exp) { float result = 1.0; for(int i=0; i<exp; i++) result *= base; return result; }</pre>
4	A correct Makefile: CC=gcc / CFLAGS=-Wall -g / all: main / main: main.o io_utils.o / [TAB] \$(CC) \$(CFLAGS) main.o io_utils.o -o main / main.o: main.c io_utils.h / [TAB] \$(CC) \$(CFLAGS) -c main.c / io_utils.o: io_utils.c io_utils.h / [TAB] \$(CC) \$(CFLAGS) -c io_utils.c / .PHONY: clean / clean: / [TAB] rm -f *.o main. Explanation: the all target triggers the build; each .o rule compiles one source file; clean removes generated files; .PHONY ensures clean always runs.
5	(a) git init — initialises a new repository in the current directory. (b) git add . then git commit -m "add Makefile project" — stages all files and creates the first commit. (c) git checkout -b feature/binary-log — creates and switches to the new branch. (d) git push -u origin feature/binary-log — pushes the branch to the remote repository named origin and sets the upstream tracking reference.

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

